Chapter 4
File Systems

- Files
- Directories
- File System Implementation
- Example File Systems

Overview

- Requirements for long term information storage:
  1. Must store large amounts of data
  2. Information stored must survive the termination of the process using it
  3. Multiple processes must be able to access the information concurrently
- Solution: Store information on disk or other external media in units called **files**.
- The **file system** is a part of operating system that manages files.

Basic Functions of File System

- Present logical (abstract) view of files and directories
  - Hide complexity of hardware devices
- Facilitate efficient use of storage devices
  - Optimize access, e.g., to disk
- Support sharing
  - Provide protection

File System

- User's view on file systems (4.1, 4.2)
  - How files are named?
  - What operations are allowed on them?
  - What the directory tree looks like?
- Implementer's view on file systems (4.3)
  - How files and directories are stored?
  - How disk space is managed?
  - How to make everything work efficiently and reliably?

Files

- Contents
  - File Naming
  - File Structure
  - File Types
  - File Access
  - File Attributes
  - File Operations
  - An Example Program Using File System Calls

File Naming (1)

- File naming - **FileName.FileExtention**
- Valid name
  - Number of characters
    - Strings of letters, digits, and special characters
    - Up to 255 characters
  - lower vs. upper case
- Extension
  - Tied to type of file
  - Used by applications

File Naming (2)

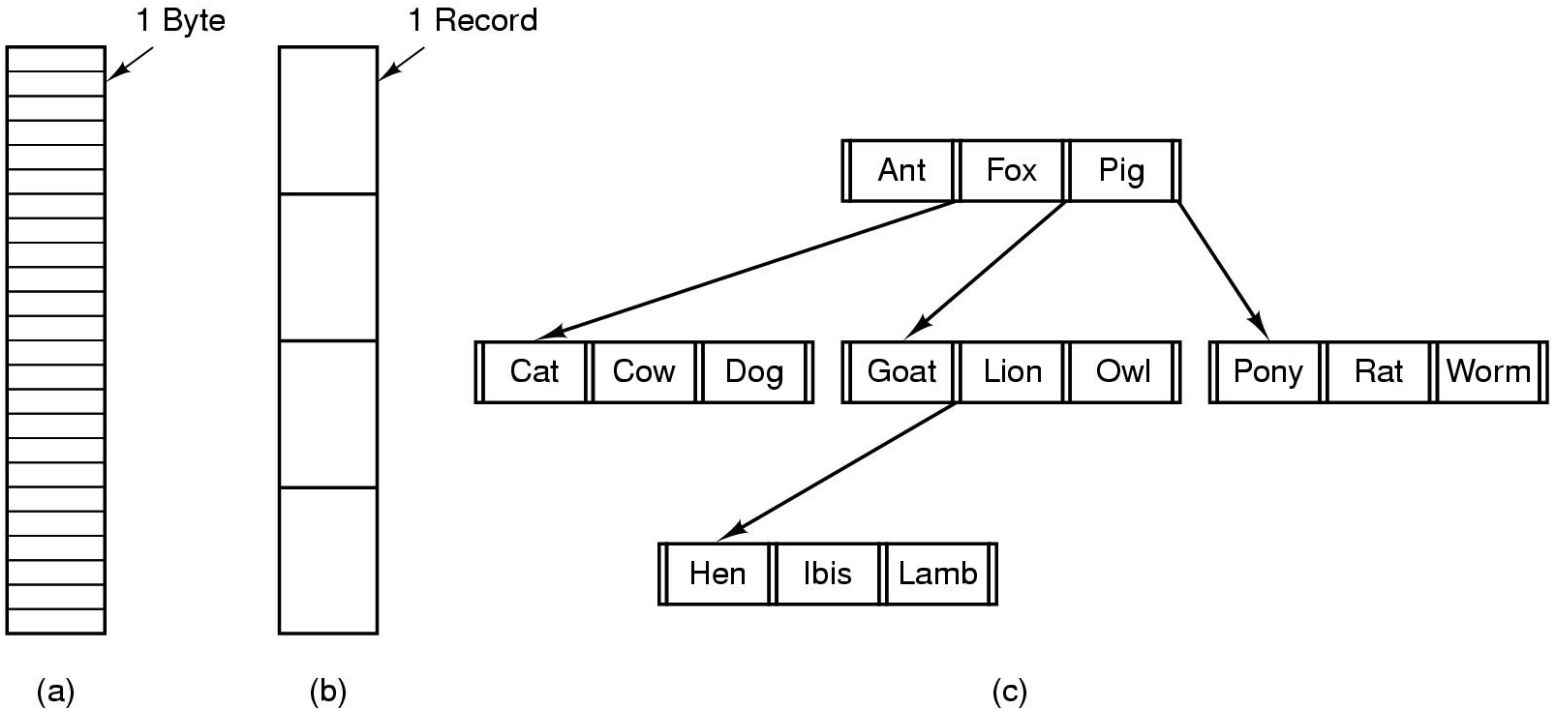
Extension	Meaning
file.bak	Backup file
file.c	C source program
file.gif	Compuserve Graphical Interchange Format image
file.hlp	Help file
file.html	World Wide Web HyperText Markup Language document
file.jpg	Still picture encoded with the JPEG standard
file.mp3	Music encoded in MPEG layer 3 audio format
file.mpg	Movie encoded with the MPEG standard
file.o	Object file (compiler output, not yet linked)
file.pdf	Portable Document Format file
file.ps	PostScript file
file.tex	Input for the TEX formatting program
file.txt	General text file
file.zip	Compressed archive

Typical file extensions

File Structure (1)

- None - sequence of words, bytes (Unix, Windows)
- Simple record structure
  - Lines
  - Fixed length
- Complex Structures
  - Record Tree
    - Records not necessarily the same length
    - Records contain keys
    - Tree sorted on key

File Structure (2)

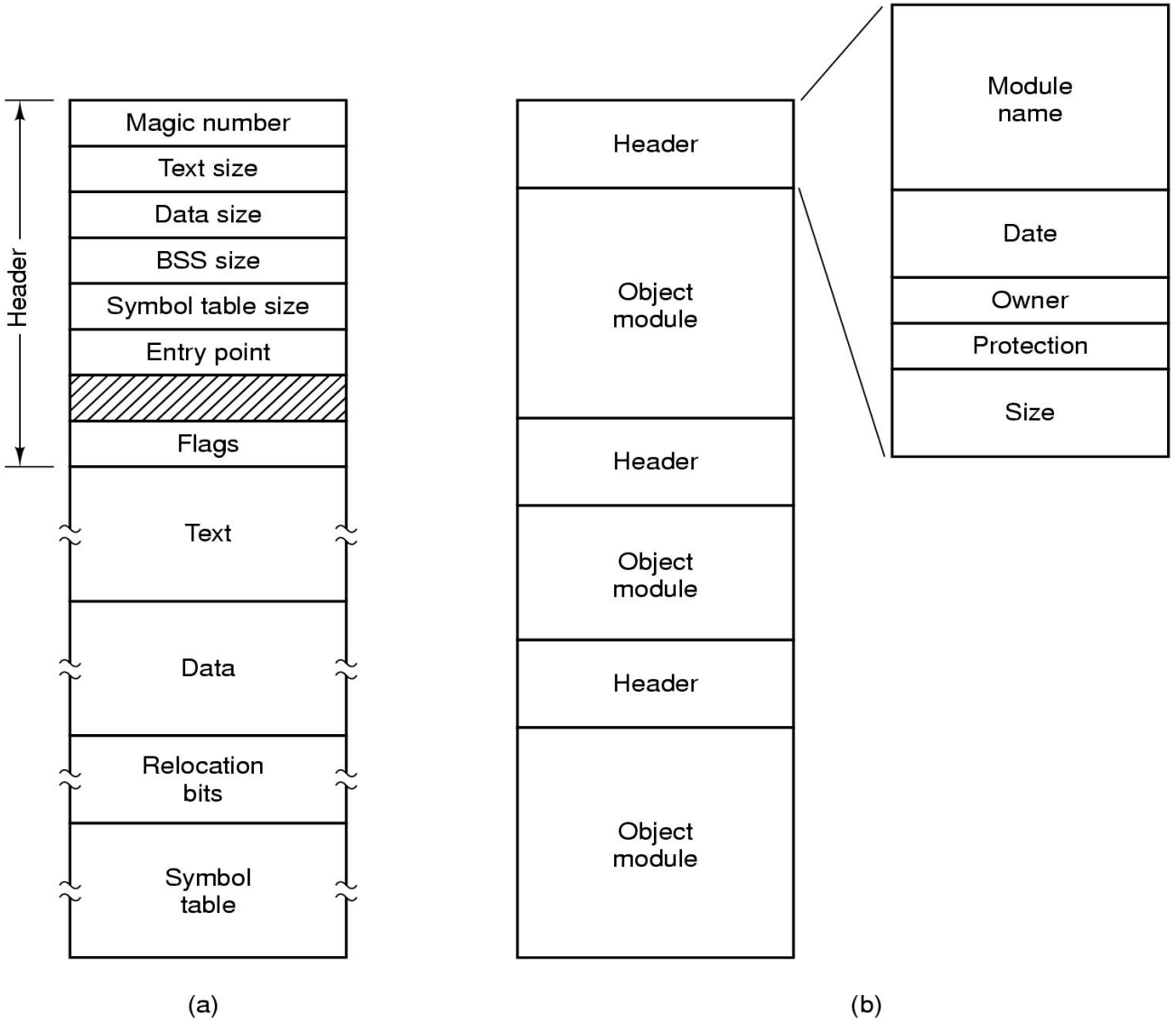


- Three kinds of files
  - byte sequence
  - record sequence
  - tree

Files types (1)

- File types
  - **Regular files** contain user information
    - ASCII file: displayed, printed, edited
    - Binary file (internal structure): executable file, archive file
  - **Directories** are system files for maintaining the structure of the file system.
  - **Character special files** are used to model serial I/O devices such as terminals, printers, and networks.
  - **Block special files** are used to model disks.

File Types (2)



(a) An executable file (b) An archive

File Access

- Sequential access
  - read all bytes/records from the beginning
  - cannot jump around, could rewind or back up
  - convenient when medium was magnetic tape
- Random access
  - bytes/records read in any order
  - essential for database systems
  - Two methods are used for specifying where to start reading.
    - *read* and then move file marker
    - move file marker (*seek*), then read (UNIX, Windows)

File Attributes

- Operating systems associate extra information with each file, called **file attributes (metadata)**.

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

Possible file attributes

File Operations

- Operations for “sequence of bytes” files

1. Create
2. Delete
3. Open
4. Close
5. Read
6. Write
7. Append
8. Seek
9. Get attributes
10. Set Attributes
11. Rename

An Example Program Using File System Calls (1)

```
/* File copy program. Error checking and reporting is minimal. */

#include <sys/types.h>                  /* include necessary header files */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]);       /* ANSI prototype */

#define BUF_SIZE 4096                   /* use a buffer size of 4096 bytes */
#define OUTPUT_MODE 0700                /* protection bits for output file */

int main(int argc, char *argv[])
{
      int in_fd, out_fd, rd_count, wt_count;
      char buffer[BUF_SIZE];

      if (argc != 3) exit(1);           /* syntax error if argc is not 3 */
```

A Unix program that copy files . [Copyfile sourcefile destfile]

An Example Program Using File System Calls (2)

```
/* Open the input file and create the output file */
in_fd = open(argv[1], O_RDONLY);     /* open the source file */
if (in_fd < 0) exit(2);              /* if it cannot be opened, exit */
out_fd = creat(argv[2], OUTPUT_MODE);  /* create the destination file */
if (out_fd < 0) exit(3);             /* if it cannot be created, exit */

/* Copy loop */
while (TRUE) {
        rd_count = read(in_fd, buffer, BUF_SIZE); /* read a block of data */
if (rd_count <= 0) break;            /* if end of file or error, exit loop */
        wt_count = write(out_fd, buffer, rd_count); /* write data */
        if (wt_count <= 0) exit(4);  /* wt_count <= 0 is an error */
}

/* Close the files */
close(in_fd);
close(out_fd);
if (rd_count == 0)                   /* no error on last read */
        exit(0);
else
        exit(5);                     /* error on last read */
}
```

Directory

- Contents
  - Single-level Directory System
  - Two-level Directory System
  - Hierarchical Directory System
  - Path Names
  - Directory Operations

Directory

- File systems have **directories** or **folders** to keep track of files
- Directory is a file containing correspondence between filenames and file locations
- Directory entries contain info about a file
  - i.e. its attributes
- Directory entries are created when the files they describe are created and removed when files are deleted

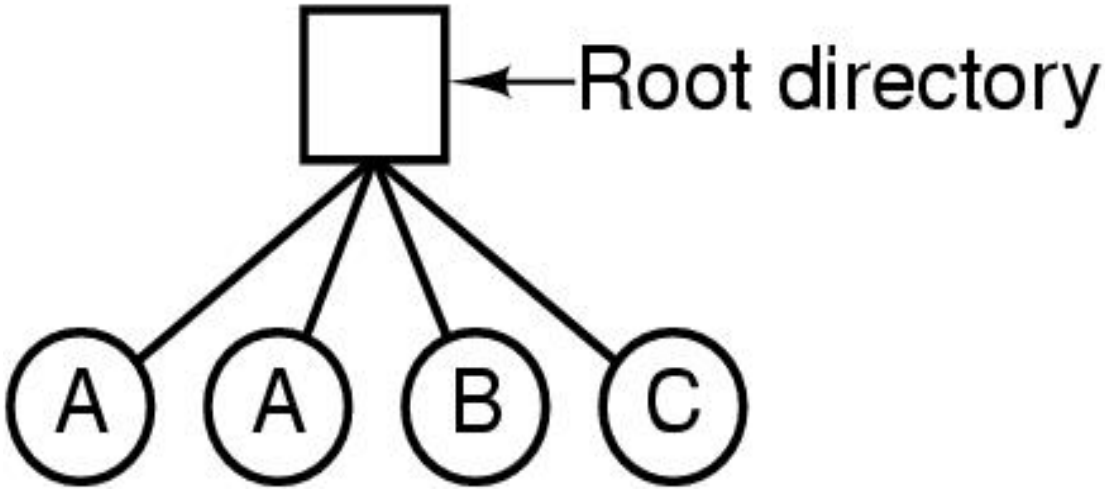
Organize the Directory to Obtain…

- Efficiency – locating a file quickly
- Naming – convenient to users
  - Two users can have same name for different files
  - The same file can have several different names
- Grouping – logical grouping of files by properties (e.g., all Java programs, all games, …)

Directory Structure

- A single-level directory has one directory (**root**) containing all the files.
- A two-level directory has a root directory and user directories.
- A hierarchical directory has a root directory and arbitrary number of subdirectories.

A single level directory system (1)



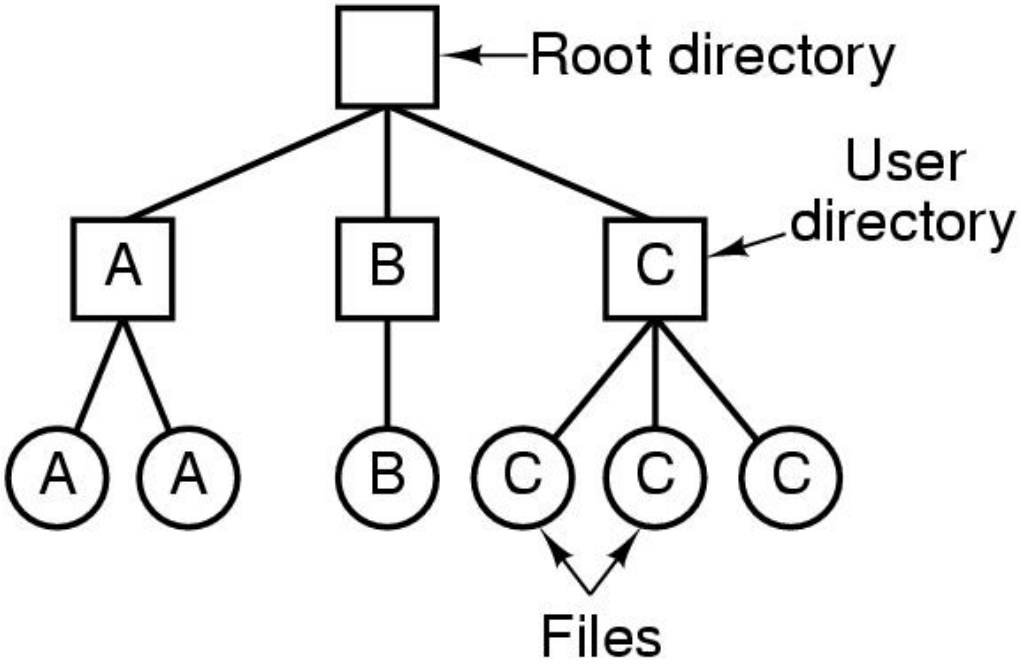
- A single level directory system
  - contains 4 files
  - owned by 3 different people, A, B, and C

A single level directory system (2)

- Advantage
  - Easy to support and understand
- Disadvantages
  - Naming Problem
    - All files must have unique names
    - Name Collision Problem
  - Grouping Problem
    - Difficult to remember file names in a large file system

Two-level Directory Systems (1)

- A user name and a file name define a path name



Letters indicate *owners* of the directories and files

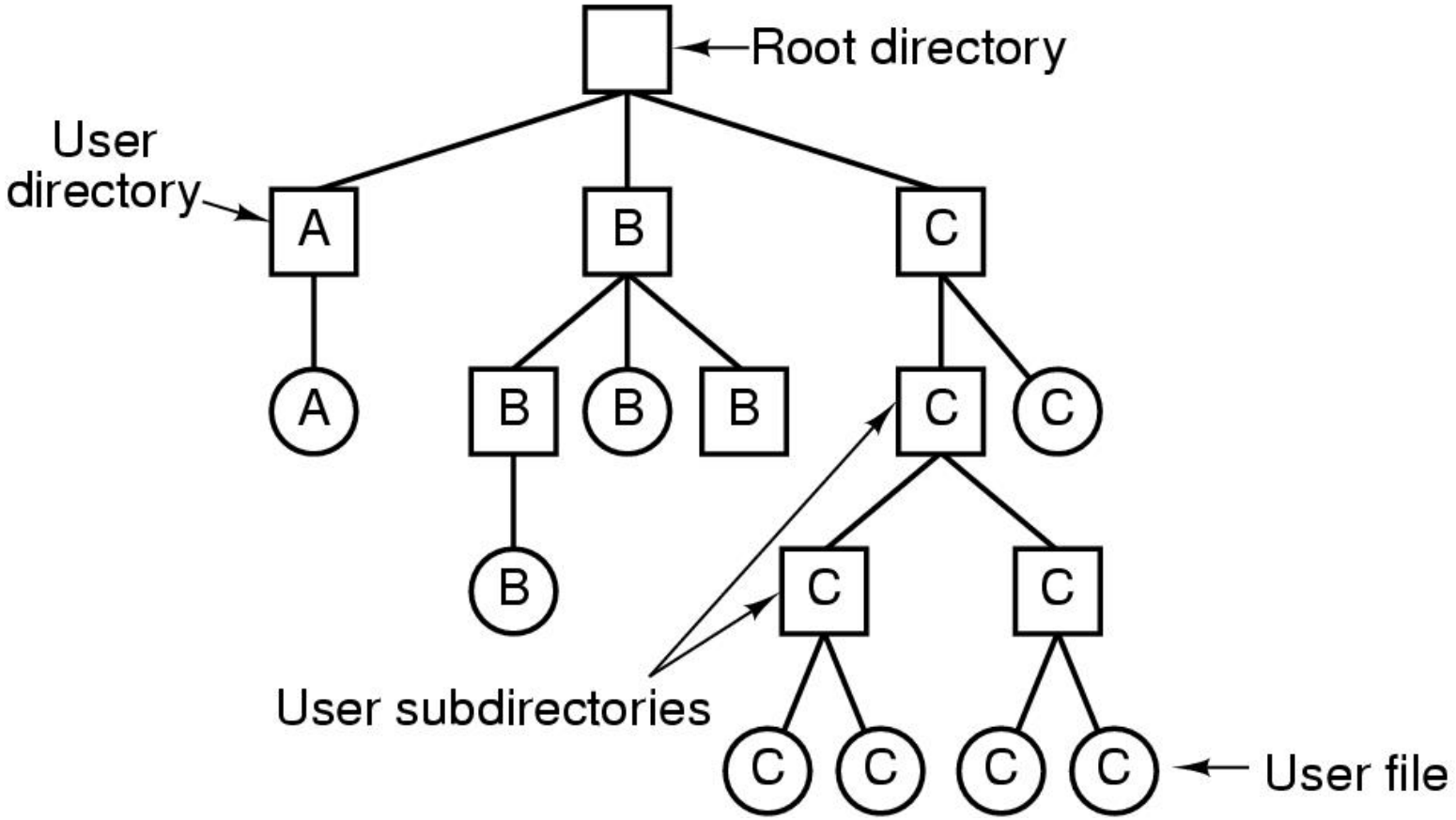
Two-level Directory Systems (2)

- Advantages
  - Resolves name-collision problem
    - Can have the same file name for different user
  - Efficient searching
  - File sharing and protection
- Disadvantage
  - Grouping problem not resolved

Hierarchical Directory Systems (1)

- Generalization of two-level directory (with arbitrary height)
- Leaf nodes are files
- Interior nodes are directories
- Each user has a current directory (working directory)
  - Can change current directory via cd command or system call
- Path names can be absolute or relative

Hierarchical Directory Systems (2)



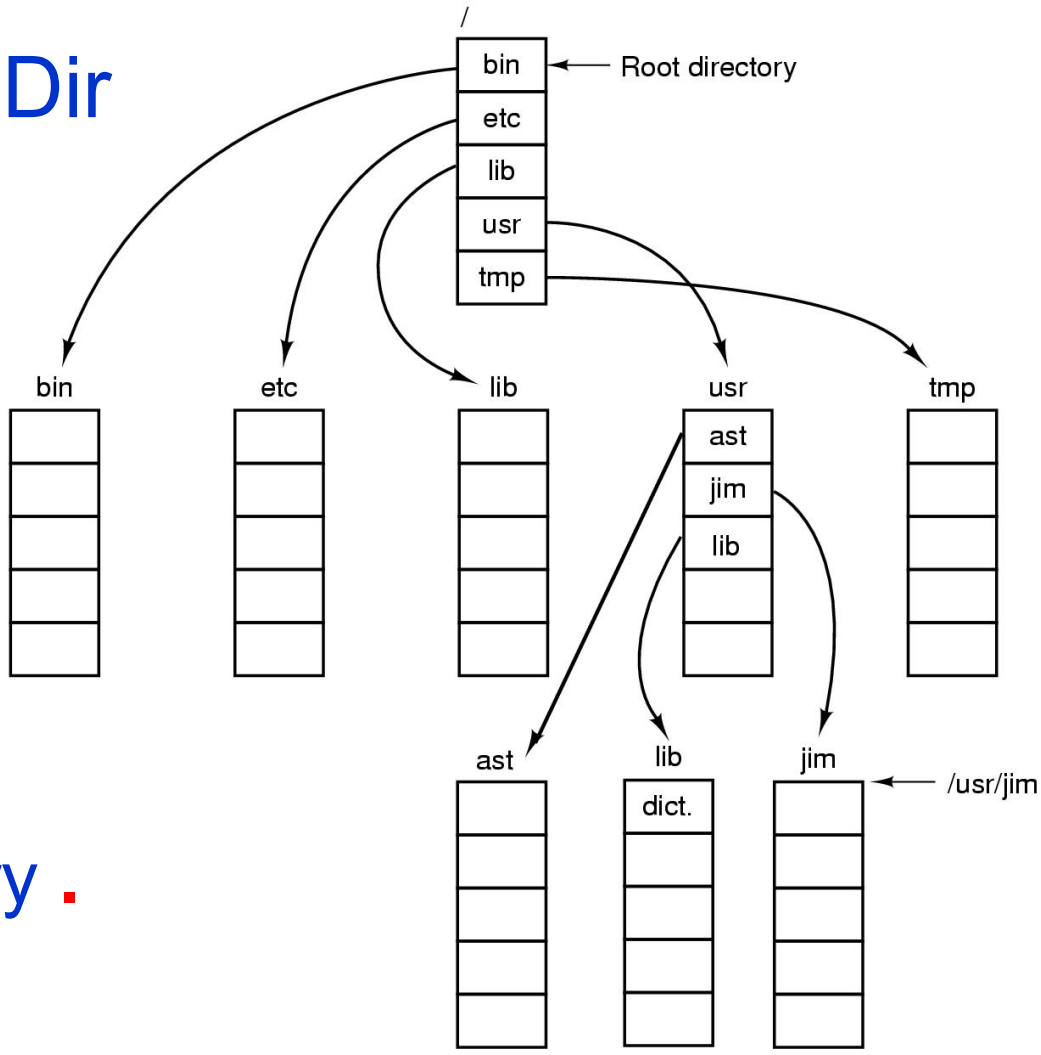
A hierarchical directory system

Path Name (1)

- Two different methods are used to specify file names in a directory tree:
  - **Absolute path name** consists of the path from the root directory to the file.
  - **Relative path name** consists of the path from the current directory (**working directory**).
- The path name would be written:
  - Winodws \usr\ast\mailbox
  - UNIX /usr/ast/mailbox
- Dot and dotdot are two special entries in the file system.
  - Dot (.) refers to the current directory.
  - Dot dot (..) refers to its parent.

Path Name (2)

- Assume working Dir
  - /usr/ast



A UNIX directory tree

- Example

  cp /usr/lib/dictionary .

  cp ../lib/dictionary .

Directory Operations

- **Create** a new directory
- **Delete** an empty directory
- **Open** a directory for reading
- **Close** a directory to release space
- **Readdir**
  - return next entry in the directory
- **Rename** a directory
- **Link**
  - Create a link from the existing file to a path name. (ie. Make a “hard link”.)
- **Unlink**
  - remove a “hard link”